David Han dlhan2@illinois.edu

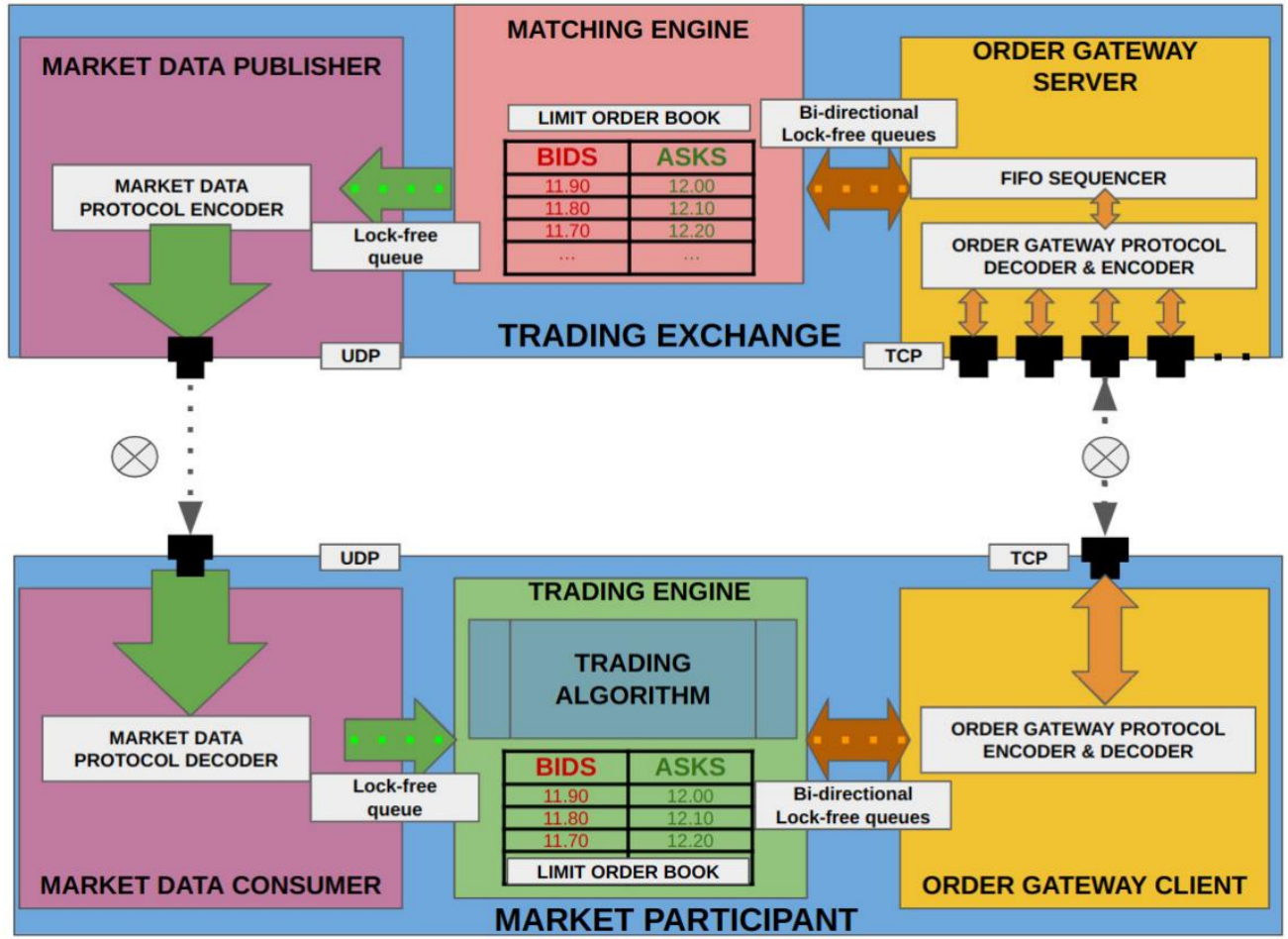
Exchange Connectivity

Resources

- Building Low Latency Applications with C++ by Sourav Ghosh
  - Chapter 7
- https://github.com/PacktPublishing/Building-Low-Latency-Applications-with-CPP
  - Chapter7/exchange/market_data
  - Chapter7/exchange/order_server
  - Chapter7/exchange/exchange_main.cpp
  - We can ignore Chapter7/exchange/matcher

Overall Design

- Today we talk about:
  - Order Gateway Server
  - Market Data Publisher



Order Gateway Server

- TCP connections with clients
- Requires a public format so market participants know how to send messages to the exchange
- FIFO Sequencer
  - Orders from different TCP connections may not arrive at FIFO sequencer chronologically
- Requires an internal format to communicate with matching engine

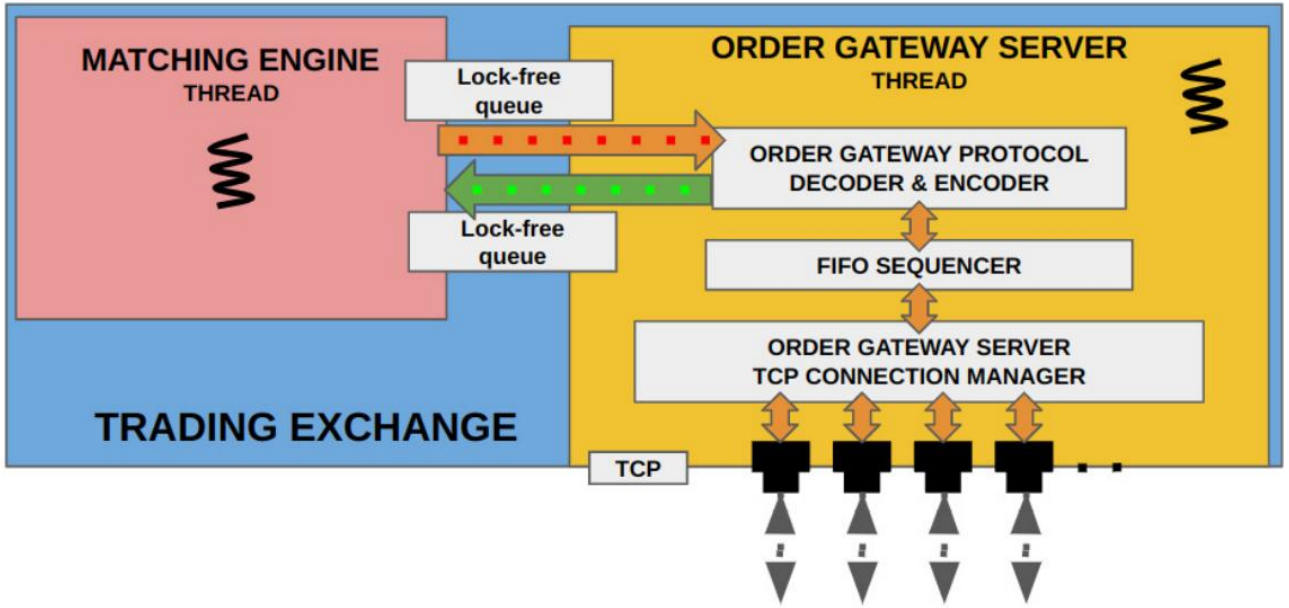


Figure 7.1 – Order gateway server and its subcomponents

Order Data Protocol – Client Request

- Public format used by clients to send order requests
- `OMClientRequest` – public format
  - `seq_num` – sequence number that uniquely identify order requests for a given client and the order it was sent
  - `MEClientRequest` – internal format

Chapter7/exchange/order_server /client_request.h

Order Data Protocol – Client Response

- Public format used by exchange to send responses to clients
- `OMClientResponse` – public format
  - `seq_num` – sequence number that uniquely identify order responses for a given client and the order it was sent
  - `MEClientResponse` – internal format

Chapter7/exchange/order_server /client_response.h

Order Server – Data

- `Common::TCPServer tcp_server_` - TCP server that accepts incoming connections and polls established connections
- `FIFOSequencer fifo_sequencer_` - gathers client requests from different TCP connections and processes them in the correct order
- `ClientResponseLFQueue outgoing_responses_` - client responses from the matching engine to be sent to connected clients
- `std::array<Common::TCPSocket *, ME_MAX_NUM_CLIENTS> cid_tcp_socket_` - hash map from `ClientId` to TCP socket
- `const std::string iface_, const int port_` - network interface and port for the TCP server
- `std::array<size_t, ME_MAX_NUM_CLIENTS> cid_next_exp_seq_num_, cid_next_outgoing_seq_num_` - next expected incoming sequence number and next outgoing sequence number
- `volatile bool run_` - starts and stops order server thread

Chapter7/exchange/order_server/order_server.h

Order Server – Constructor

- Set up pointers to lock-free queues
- Set all expected incoming and outgoing sequence numbers to `1`
- Set all TCP sockets to `nullptr`
- Set up `recv_callback_` callback member in TCP server pointing to `recvCallback()`
- Set up `recv_finished_callback_` callback member in TCP server pointing to `recvFinishedCallback()`

Chapter7/exchange/order_server/order_server.cpp

Order Server – start(), stop(), Destructor

- Start
  - Set `run_ = true`
  - TCP server starts listening for connections on given interface and port
  - Start thread which runs `run()`
- Stop
  - Set `run_ = false`
  - This will cause the thread to exit its main loop, then closing the thread
- Destructor
  - Runs `stop()`
  - Sleeps for 1 second to finish any pending tasks

Chapter7/exchange/order_server/order_server.cpp

Order Server – recvCallback()

- Receives parameters socket pointer and timestamp when kernel received the packet
- Cast chunks of `sizeof(OMClientRequest)` bytes from socket buffer into `OMClientRequest` objects
- If it's the first request from a client, update `cid_tcp_socket_`, the client ID to socket hash map
- Check that the client is using the expected socket
- Check that we got the expected sequence number
- Increment expected sequence number
- Add our new `OMClientRequest` and timestamp to the FIFO sequencer

Chapter7/exchange/order_server/order_server.h

Order Server – recvFinishedCallback()

- Tells the FIFO sequencer to sort all the new `OMClientRequests` and send `MEClientRequests` to the matching engine

Chapter7/exchange/order_server/order_server.h

FIFO Sequencer – Data

- `ClientRequestLFQueue *incoming_requests_` - client requests to be sent to matching engine
- `std::array<RecvTimeClientRequest, ME_MAX_PENDING_REQUESTS> pending_client_requests_` - a place to store and sort client requests before they are sent to `incoming_requests_`
- `RecvTimeClientRequest` is defined as a timestamp and `MEClientRequest` object

FIFO Sequencer – addClientRequest()

- Check if there are too many pending requests
- Add a client request to `pending_client_requests_` at `pending_size_` position
- Increment `pending_size`

Chapter7/exchange/order_server/fifo_sequencer.h

FIFO Sequencer – sequenceAndPublish()

- Sort all `RecvTimeClientRequest` in `pending_client_requests_` by time
  - We can do this because we implemented operator overload for <
  - We rarely expect `pending_size_` to be large, so it's fine to use `std::sort()`
- Add all `MEClientRequests` to `incoming_requests_`
- Set `pending_size_` = 0

Sending Client Responses – run()

- Main loop for the order server thread
- Repeatedly calls `tcp_server_.poll()` and `tcp_server_.sendAndRecv()` to poll for new connections and new messages in socket buffers
- If there are new `MEClientResponses` in `outgoing_responses_`:
  - Send the outgoing sequence number and `MEClientResponse` to the corresponding socket given the client ID
    - We can safely do this because there is no padding between sequence number and `MEClientResponse`
  - Increment outgoing sequence number for the given client

Chapter7/exchange/order_server/order_server.h

Market Data Publisher

- Receive market updates from matching engine
- Market data protocol encoder
  - Converts internal format to public protocol
- Snapshot synthesizer
  - Gathers market updates and publishes order book state
  - Runs on separate thread to not slow down the market data publisher thread
- 2 UDP multicast streams
  - Incremental stream
  - Snapshot stream

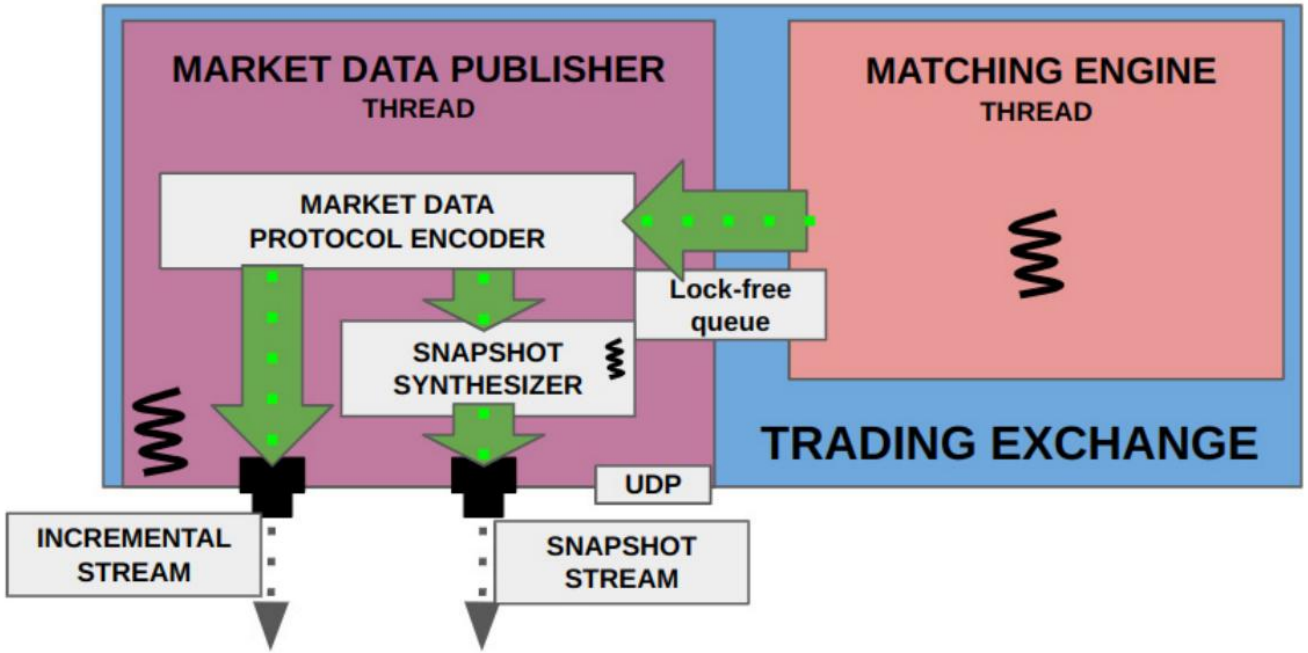


Figure 7.2 – Market data publisher and its subcomponents

Market Data Protocol

- Public format that the exchange uses to send market data
- `MDPMarketUpdate` – public format
  - `seq_num` – sequence number that uniquely identify market data and the order it was sent
  - `MEMarketUpdate` – internal format
    - `MarketUpdateType` has 2 new types – `SNAPSHOT_START` and `SNAPSHOT_END`

Chapter7/exchange/market_data/market_update.h

Market Data Publisher – Data

- `size_t next_inc_seq_num_` - maintains outgoing sequence number
- `MEMarketUpdateLFQueue *outgoing_md_updates_` - market updates from the matching engine to be sent to clients
- `MDPMarketUpdateLFQueue snapshot_md_updates_` - market updates to be send to snapshot synthesizer since it runs on different thread
- `Common::McastSocket incremental_socket_` - socket to publish UDP messages on incremental multicast stream
- `SnapshotSynthesizer *snapshot_synthesizer_`
- `volatile bool run_` - starts and stops MarketDataPublisher thread

Chapter7/exchange/market_data/market_data_publisher.h

Market Data Publisher – Constructor

- Set up pointers to lock-free queues
- Initialize UDP multicast socket `incremental_socket_` using `incremental_ip` and `incremental_port`
- Create snapshot synthesizer and pass `snapshot_md_updates_` and snapshot multicast stream information from `snapshot_ip`, `snapshot_port`

Chapter7/exchange/market_data/market_data_publisher.cpp

Market Data Publisher – start(), stop(), Destructor

- Start
  - Set `run_ = true`
  - Start thread which runs `run()`
  - Calls `snapshot_synthesizer_->start()` which creates separate thread for snapshot synthesizer
- Stop
  - Set `run_ = false`
  - This will cause the thread to exit its main loop, then closing the thread
  - Calls `snapshot_synthesizer_->stop()` which closes the snapshot synthesizer thread
- Destructor
  - Runs `stop()`
  - Sleeps for 5 seconds to finish any pending tasks
  - Frees `snapshot_synthesizer_`

Chapter7/exchange/market_data/market_data_publisher.h

Publishing Order Book Updates – run()

- Main loop for the order server thread
- If there are new `MEMarketUpdates` in `outgoing_md_updates_`:
  - Copy the outgoing sequence number and `MEMarketUpdate` to the incremental socket outgoing buffer
    - We can safely do this because there is no padding between sequence number and `MEMarketUpdates`
  - Push the `MDPMarketUpdate` object consisting of sequence number and `MEMarketUpdate` object to `snapshot_md_updates_`
  - Increment outgoing sequence number
- Publish outgoing data in the socket buffer

Chapter7/exchange/market_data/market_data_publisher.cpp

Snapshot Synthesizer – Data

- `MDPMarketUpdateLFQueue *snapshot_md_updates_` - same as Market data publisher
- `McastSocket snapshot_socket_` - socket to publish UDP messages on snapshot multicast stream
- `std::array<std::array<MEMarketUpdate *, ME_MAX_ORDER_IDS>, ME_MAX_TICKERS> ticker_orders_` - `ticker_orders_[i][j]` points to a `MEMarketUpdate` object with ticker ID = i and order ID = j
- `MemPool<MEMarketUpdate> order_pool_` - memory pool to store `MEMarketUpdate` objects that `ticker_orders_` will point to
- `size_t last_inc_seq_num_` - sequence number of the last market update received
- `Nanos last_snapshot_time_` - when the last snapshot was published
- `volatile bool run_` - starts and stops `MarketDataPublisher` thread

Chapter7/exchange/market_data/snapshot_synthesizer.h

Snapshot Synthesizer – Constructor

- Set up pointers to lock-free queues
- Initialize UDP multicast socket `snapshot_socket_` using `snapshot_ip` and `snapshot_port`
- Initializes `order_pool_` with size `ME_MAX_ORDER_IDS`
- Create snapshot synthesizer and pass `snapshot_md_updates_` and snapshot multicast stream information from `snapshot_ip`, `snapshot_port`
- Fill 2D array `ticker_orders_` with `nullptr`

Chapter7/exchange/market_data/snapshot_synthesizer.cpp

Snapshot Synthesizer – start(), stop(), Destructor

- Start
  - Is called by `MarketDataPublisher::start()`
  - Set `run_ = true`
  - Start snapshot synthesizer thread which runs `run()`
- Stop
  - Is called by `MarketDataPublisher::stop()`
  - Set `run_ = false`
  - This will cause the thread to exit its main loop, then closing the thread
- Destructor
  - Runs `stop()`

Chapter7/exchange/market_data/snapshot_synthesizer.cpp

Snapshot Synthesizer – addToSnapshot()

- If market update type is `ADD`
  - Allocate our `MEMarketUpdate` object to memory pool
  - `ticker_orders_[ticker ID][order ID]` should point to our new memory pool allocation
- If market update type is `MODIFY`
  - Modify price and quantity fields from `*ticker_orders_[ticker ID][order ID]`
- If market update type is `CANCEL`
  - Deallocate our `MEMarketUpdate` object from memory pool
  - `ticker_orders_[ticker ID][order ID]` now points to `nullptr`
- We ignore all other market update types since they do not affect order book state

Chapter7/exchange/market_data/snapshot_synthesizer.cpp

addToSnapshot – publishSnapshot()

- Send the following `MDPMarketUpdate` messages to `snapshot_socket_`:
  - The first message has type `MarketUpdateType::SNAPSHOT_START` to indicate start of snapshot messages
    - Order ID is `last_inc_seq_num_`
  - For each instrument/ticker ID:
    - Send a message with type `MarketUpdateType::CLEAR` to instruct clients to clear their order book
    - Send a message for any element in ticker_orders[ticker ID] that is not nullptr
  - The last message has type `MarketUpdateType::SNAPSHOT_END` to indicate end of snapshot messages
    - Order ID is `last_inc_seq_num_`
- For each message:
  - Increment the sequence number start from `1`

Chapter7/exchange/market_data/snapshot_synthesizer.cpp

Purpose of `last_inc_seq_num_`

- From the client's perspective:
  - The client needs a way to know which incremental market updates are/aren't included in the snapshot
- We need to store the "dividing" sequence number somewhere
- We cannot store it in the intuitive SeqNum field because it must be sequential
- Choose Order ID field instead

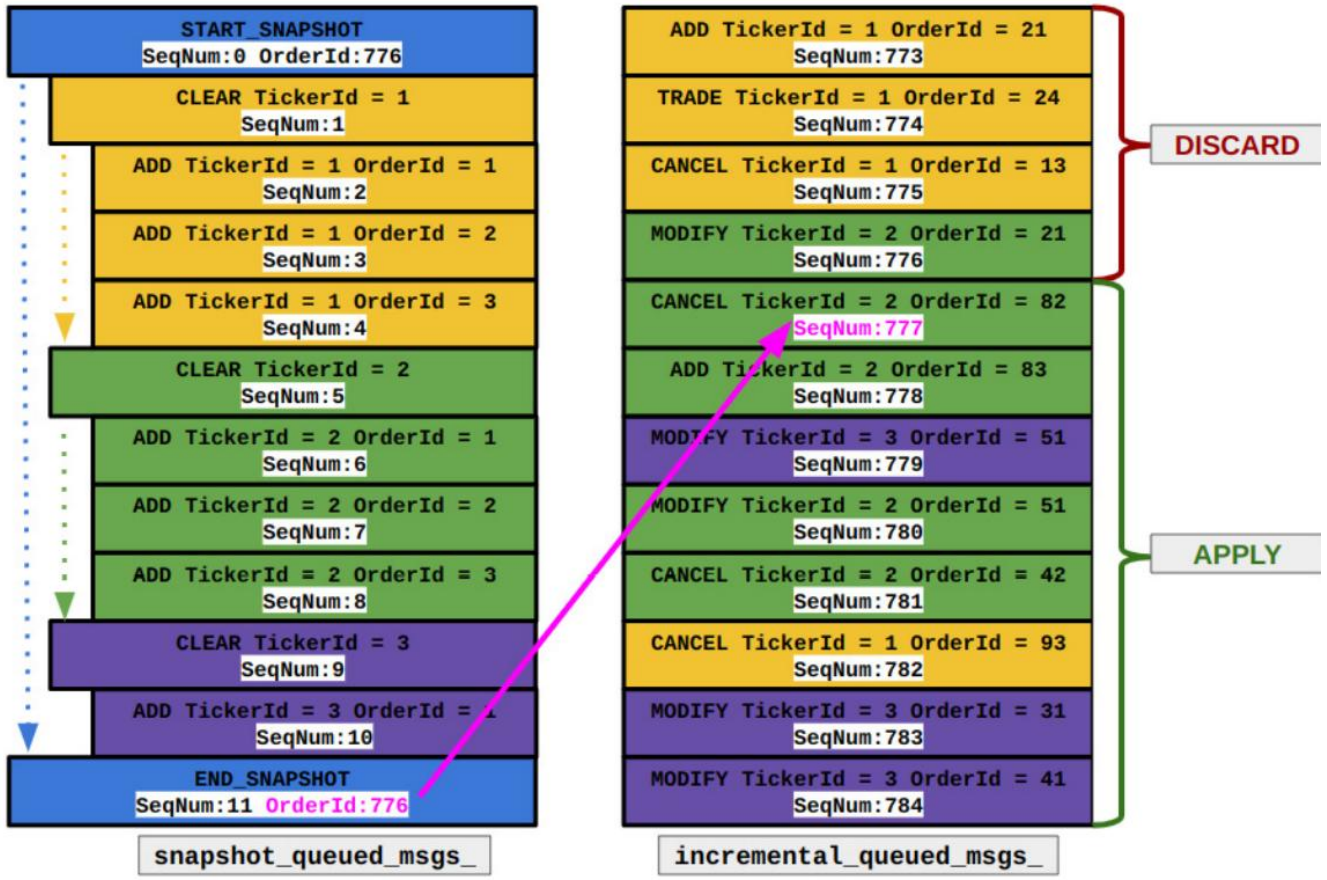


Figure 8.2 – Example state of snapshot and incremental queues when recovery is possible

Snapshot Synthesizer – run()

- Main loop for the snapshot synthesizer thread
- If there are new `MDPMarketUpdates` in `snapshot_md_updates_`:
  - Add the `MDPMarketUpdate` object to snapshot by calling `SnapshotSynthesizer:: addToSnapshot()`
- If it has been more than 60 ns since last snapshot was published, publish a snapshot by calling `SnapshotSynthesizer::publishSnapshot()`

Chapter7/exchange/market_data/snapshot_synthesizer.cpp

Exchange Application

- Create `MatchingEngine`, `MarketDataPublisher`, `OrderServer`, `Logger` instances
- Signal handler to free the above when Unix signal SIGINT is sent (ctrl + C in shell)
- Set up 3 lock-free queues of type `ClientRequestLFQueue`, `ClientResponseLFQueue`, `MEMarketUpdateLFQueue`
- Network interface, IPs, ports can be arbitrary as long as it matches the clients
- Finally, let the main thread sleep indefinitely since the other threads will do the actual work

Questions?

- Next week: Client Connectivity